CoursePilot API Specification

Base URL: [insert URL of CoursePilot here]

1. Introduction

This specification ensures that when the AI extracts a "Homework" assignment, the Dashboard knows exactly how to display it (including its color tag, its hard deadline, and its checklist of sub-tasks).

Design Principles

- **RESTful Design:** Standard HTTP methods (GET, POST, PATCH).
- **Schema Parity:** API responses strictly mirror the Firestore Database Schema.
- **Authentication:** All requests must include a Firebase Auth Token in the header.

2. Authentication (The Gatekeeper)

Concept

To ensure privacy, the Backend never accepts a request unless it knows exactly who sent it. We do not handle passwords directly; instead, we use a secure "access badge" (Token) issued by Google.

Technical Specification

- **Mechanism:** Bearer Token (JWT).
- **Header Required:** Authorization: Bearer <firebase_jwt_token>
- **Behavior:** If the token is missing or expired, the API will return 401 Unauthorized.

3. Feature: Syllabus Upload

Concept: The Trigger

The actual file upload happens on the Frontend (sending the PDF directly to Google's cloud storage). This API endpoint is the **notification trigger**. It tells the Backend: *"Hey, the user just finished uploading a syllabus for CSCI 570. Please start the AI processing pipeline."*

We also allow the user to provide metadata (like the Semester or Instructor) right at the start, so we don't have to guess it later.

Technical Specification

- **Endpoint:** POST /api/v1/syllabi/upload
- **Action:** Triggers a Redis Job for AI parsing.

Request Body:

JSON
```
{
  "course_code": "CSCI 570",
  "file_name": "CSCI570_Syllabus.pdf",
  "storage_path": "users/abc123/syllabi/CSCI570.pdf",
  "metadata": {
     "semester": "Spring 2026",
     "instructor": "Dr. Shamsian"
  }
}
```

Response (202 Accepted): *Note: We use "202 Accepted" instead of "200 OK" to indicate that the request was received, but the heavy AI work is happening in the background.*

JSON
```
{
  "message": "Syllabus received. Processing pipeline started.",
  "syllabus_id": "syl_abc12345",
  "job_id": "job_xyz789",
  "status": "PROCESSING",
  "uploaded_at": "2026-02-05T14:30:00Z"
}
```

4. Feature: The Calendar Dashboard

Concept: The View

This is the core of the application. The Dashboard needs to request all events to populate the calendar.

Key Data Updates (from Database Schema):

1. **Deadlines vs. Work Blocks:** We now distinguish between when a student *plans* to do the work (start_time/end_time) and the actual hard deadline (due_date).

2. **Visual Styling:** The backend sends a color_code (e.g., Teal for Homework, Red for Exams) so the frontend doesn't need complex logic to guess colors.
3. **Checklists:** An event isn't just a block; it contains sub-tasks (next_steps) like "Read Chapter 4" or "Submit Draft."

Technical Specification

- **Endpoint:** GET /api/v1/events
- **Query Params:** ?start_date=2026-02-01&end_date=2026-03-01 (Optional filtering).

Response (200 OK):

JSON

```
{
  "data": [
    {
      "id": "evt_55566",
      "title": "Assignment 1",
      "course_code": "CSCI 570",
      "event_type": "HW",
      "status": "CONFIRMED",

      // TIMING LOGIC
      // start/end = The visual block on the calendar (When am I working?)
      "start_time": "2026-02-20T14:00:00Z",
      "end_time": "2026-02-20T16:00:00Z",

      // due_date = The hard deadline (When is it actually due?)
      "due_date": "2026-02-20T23:59:00Z",
      "all_day": false,

      // UI STYLING & METADATA
      "color_code": "#44BDD0",
      "tags": ["Urgent", "CS402"],
      "description": "Complete problems 1-5.",
      "location": "Brightspace",
      "weight_percentage": 5,

      // CHECKLIST (Sub-tasks)
      "next_steps": [
        { "text": "Read Chapter 4", "is_completed": true },
        { "text": "Draft solutions", "is_completed": false }
      ],

      "source_link": "gs://bucket/users/abc/syllabus.pdf#page=2"
```

```
    }
  ],
  "count": 1
}
```

5. Feature: Editing & Human-in-the-Loop

Concept: Correction

AI is not perfect. It might read a date wrong, or a student might want to add a personal "Urgent" tag to an exam. This endpoint allows the user to modify an event.

Most importantly, this is used to transition an event from **PENDING_REVIEW** (AI found it) to **CONFIRMED** (User accepted it).

Technical Specification

- **Endpoint:** PATCH /api/v1/events/{event_id}
- **Behavior:** Partial updates. You only need to send the fields you want to change.

Request Body (Scenario: User fixing a date and checking off a task):

```
JSON
{
  "status": "CONFIRMED",
  "due_date": "2026-02-21T23:59:00Z",  // Fixing a date hallucination
  "tags": ["Urgent", "Reviewed"],      // Adding custom tags
  "next_steps": [                      // Updating checklist progress
     { "text": "Read Chapter 4", "is_completed": true },
     { "text": "Draft solutions", "is_completed": true },
     { "text": "Submit to Gradescope", "is_completed": false }
  ]
}
```

Response (200 OK):

```
JSON
{
  "id": "evt_55566",
  "status": "CONFIRMED",
  "updated_at": "2026-02-06T10:30:00Z",
  "message": "Event updated successfully."
```

}

6. Data Dictionary (Enums)

Concept: Shared Vocabulary

To prevent bugs, the Frontend and Backend must agree on the exact spelling of specific categories.

Technical Definitions

EventType:

- Exam: Major tests, midterms, finals.
- HW: Homework, assignments, problem sets.
- Lecture: Class sessions or labs.

EventStatus:

- PENDING_REVIEW: The AI created this event, but the user hasn't looked at it yet.
- CONFIRMED: The user has verified this event is accurate.

SyllabusStatus:

- UPLOADED: File sits in storage.
- PROCESSING: AI is currently reading it.
- PENDING_REVIEW: AI finished; events are waiting for approval.
- CONFIRMED: All events approved; syllabus is "Live".
- FAILED: Something went wrong (e.g., corrupted PDF).